LEAD DEV MOBILE • FREELANCE

Frédéric Bellano

Développeur mobile et formateur

Me contacter

Teams

frederic.bellano@ext.eduservices.org

13 ans dans le développement mobile

9 ans dans la formation

2022 début de mon activité de freelance

WORLDLINE

Mon expérience principale

L'essentiel de mon parcours professionnel s'est construit chez Worldline.

Mobile
Java · Kotlin
Objective-C · Swift

Backend
Développement de services

PWA
Vue.js

Web
Angular

RETAIL

Une application à fort impact métier

Participation au développement d'une application générant plus de **1 M€ de chiffre d'affaires par semaine**.

TRANSPORT

Transport de marchandises

Développement d'une application métier dans ce secteur.

Applications mobiles et connectées

Construire une application mobile qui dialogue avec des capteurs virtuels et la faire évoluer pendant tout le module.

Flutter · MVVM w/ Clean Architecture · IoT

Un module de **45 h 30**

Compétence visée

À la fin du module, vous devrez pouvoir :

> Architecturer, développer et déployer une application mobile multiplateforme qui dialogue avec des objets connectés, en tenant compte de la sécurité, des performances et de la maintenabilité.

Le programme en 4 étapes

1. **Construire** l'application mobile et ses contrats.
2. **Connecter** l'application, le broker et les objets simulés.
3. **Fiabiliser** le temps réel, les données et la sécurité.
4. **Livrer** une solution testée, observable et déployable.

```
Flutter → Architecture → Protocoles → IoT → Résilience → Livraison
```

1. Construire — séances 1 à 3

Séance	Contenu	Résultat
1	Flutter, Freezed et contrats MVVM	Interface statique et Repository simulé
2	ViewModel, états scellés, Provider et tests	Application structurée et testable
3	Simulateur IoT, topics MQTT et JSON	Contrats de messages documentés

À ce stade, l'application est structurée et le simulateur produit ses premiers messages.

2. Connecter — séances 4 à 6

Séance	Contenu	Résultat
4	Mosquitto, publish/subscribe, QoS et LWT	Broker local opérationnel
5	Flotte simulée, commandes et pannes	Scénarios IoT reproductibles
6	Repository MQTT, streams et reconnexion	Mesures reçues dans Flutter

À ce stade, l'application communique avec une flotte d'objets virtuels.

3. Fiabiliser — séances 7 à 10

Séance	Contenu	Résultat
7	Graphiques, fraîcheur et alertes locales	Dashboard temps réel
8	Cache, retry et commandes différées	Mode offline-first
9	Backend HTTP et InfluxDB	Historique des mesures
10	TLS, comptes, ACL, RGPD et ETSI	Architecture durcie

L'application doit continuer à donner une information utile lorsque le réseau devient instable.

4. Livrer — séances 11 à 13

Séance	Contenu	Résultat
11	Tests intégrés, logs, métriques et traces	Système observable
12	Build Android/iOS, CI et Docker Compose	Livraison reproductible
13	Challenge de panne et soutenance	Projet final démontré

13 séances × 3 h 30 = 45 h 30

Les jalons du projet

Quand ?	Ce que nous devons pouvoir montrer
Après 7 h	Une application Flutter structurée en MVVM
Après 21 h	Flutter reçoit plusieurs capteurs simulés par MQTT
Après 35 h	Le système est temps réel, offline et sécurisé
Après 45 h 30	Le produit est testé, déployable et présenté

Une approche software-first

Tout le parcours obligatoire fonctionne avec :

- un émulateur Android, un simulateur iOS ou un téléphone de test ;
- un simulateur de capteurs fourni par le formateur ;
- Mosquitto en local ;
- un backend et InfluxDB en local ;
- des scénarios de panne reproductibles.

Aucun achat de matériel et aucun cloud payant ne sont nécessaires.

Le simulateur remplace le matériel

Chaque instance se comporte comme un objet connecté :

- il possède un identifiant ;
- il publie température, humidité et batterie ;
- il reçoit des commandes ;
- il annonce sa connexion ou sa déconnexion ;
- il peut simuler une panne ou une valeur anormale.

```
Objets simulés → Mosquitto → SensorHub
```

Si l'établissement fournit ensuite quelques cartes, un objet physique pourra remplacer une instance simulée sans modifier l'architecture.

Notre méthode

Chaque séance suivra autant que possible le même rythme :

1. comprendre une notion et son intérêt ;
2. observer une démonstration courte ;
3. réaliser un TP guidé ;
4. poursuivre en autonomie ;
5. vérifier le résultat avec un test ou une démonstration ;
6. conserver une version fonctionnelle avant de continuer.

Ce que j’attends de vous

- poser des questions dès qu’un point bloque ;
- exécuter régulièrement l’application ;
- lire les messages d’erreur avant de modifier le code ;
- travailler par petites étapes ;
- savoir expliquer les choix réalisés ;
- participer aux revues et aux démonstrations.

Le résultat compte. La compréhension du chemin qui y mène compte tout autant.

Le fil rouge : SensorHub

Une application de supervision de capteurs connectés.

SensorHub, notre projet fil rouge

SensorHub est une application mobile de supervision d'une flotte de capteurs simulés.

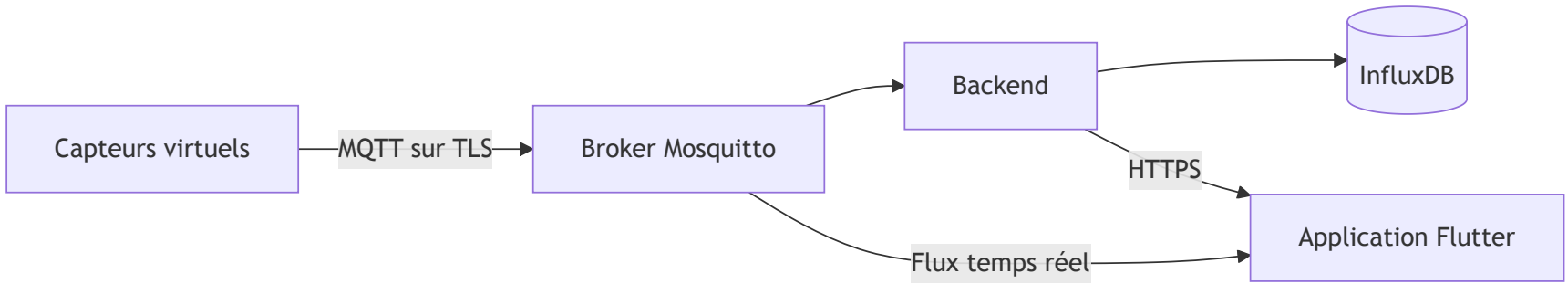
Elle permettra de :

- visualiser la température, l'humidité, la batterie et la connectivité ;
- recevoir les mesures en temps réel avec MQTT ;
- détecter une anomalie et déclencher une alerte ;
- consulter l'historique des mesures ;
- continuer à fonctionner pendant une coupure réseau ;
- sécuriser et observer les échanges.

```
Données locales → Simulateur + MQTT → Temps réel → Offline → Sécurité → Livraison
```

À chaque séance, nous ajouterons une capacité tout en conservant une application fonctionnelle.

Architecture finale simplifiée



Tout peut être exécuté localement sur un ordinateur.

Objectif des 7 premières heures

Une application Flutter fonctionnelle, testable et prête à évoluer.

À la fin de cette étape

Vous saurez :

- composer un écran avec des widgets ;
- faire évoluer l'interface à partir d'un état ;
- organiser le code avec MVVM ;
- utiliser Provider et `ChangeNotifier` ;
- isoler les données derrière un Repository ;
- représenter les états Loading, Content et Error ;
- tester un ViewModel.

Flutter en quelques mots

Flutter est un SDK permettant de créer des applications mobiles avec une seule base de code.

- le langage utilisé est **Dart** ;
- l'interface est composée de **widgets** ;
- le même projet cible **Android et iOS** ;
- le **hot reload** permet de visualiser rapidement les modifications ;
- les API natives restent accessibles lorsque l'application en a besoin.

```
Code Dart + widgets Flutter
            ↓
    Application mobile
       Android · iOS
```

Dans ce module, nous utiliserons Flutter uniquement pour développer une application mobile.

Flutter et Dart

```
Dart
└── le langage, son typage, ses classes et son asynchronisme

Flutter
└── le SDK, les widgets, le rendu et les outils de développement
```

Nous apprendrons Dart au fur et à mesure des besoins de l'application.

L’écosystème de packages : pub.dev

pub.dev est le catalogue de référence des packages Dart et Flutter.

Avant d’ajouter un package, vérifiez :

- sa compatibilité avec Dart, Flutter, Android et iOS ;
- sa documentation et ses exemples ;
- sa date de dernière publication et son changelog ;
- son score, ses pub points et son adoption ;
- sa licence, son dépôt et les problèmes ouverts ;
- la version compatible avec votre projet.

> Un package populaire n’est pas automatiquement adapté à notre besoin.

Pourquoi Flutter ?

- **Une base de code commune** pour Android et iOS.
- **Une interface cohérente** et entièrement composable avec des widgets.
- **Une boucle de développement rapide** grâce au hot reload.
- **Un modèle réactif** adapté aux flux de données des capteurs.
- **Un accès aux capacités mobiles** : notifications, réseau et stockage sécurisé.
- **Un outillage intégré** : analyse statique, tests et DevTools.
- **Un pattern d'architecture de présentation lisible** avec MVVM.

Et les autres solutions ?

Solution	Point fort	Positionnement dans ce module
Flutter	UI cohérente et multiplateforme	Solution retenue
React Native	Écosystème JavaScript/React	Alternative pertinente pour une équipe maîtrisant cet écosystème
Kotlin Multiplatform	Partage du métier et intégration native	Alternative adaptée aux équipes proches des plateformes natives

Ces trois solutions sont valables. Pour aller jusqu'à l'IoT, la sécurité et le déploiement en 45 h 30, nous devons en approfondir une seule.

Les compromis de Flutter

Flutter ne supprime pas les différences entre les plateformes :

- certaines permissions restent spécifiques à Android ou iOS ;
- les intégrations natives avancées demandent une configuration spécifique ;
- un package Flutter ne donne pas toujours accès à toutes les fonctionnalités natives d'Android et d'iOS ;
- il faut parfois écrire ou diagnostiquer du code natif ;
- le comportement doit être testé sur de vrais appareils.

> Multiplateforme ne signifie pas « écrire une fois et ne plus jamais penser aux plateformes ».

Le choix de Flutter est posé. Voyons maintenant comment il construit une interface.

Préparer votre poste

Objectif : disposer d'un environnement Flutter et d'au moins une cible mobile fonctionnelle.

Pour tous

- installer le SDK Flutter stable ;
- ajouter Flutter au `PATH` ;
- installer VS Code ;
- ajouter les extensions Flutter et Dart.

Cible Android

- installer Android Studio ;
- installer le SDK Android et ses outils ;
- créer un émulateur ou connecter un téléphone ;
- accepter les licences Android.

Cible iOS · macOS

- installer la dernière version de Xcode ;
- configurer les Command Line Tools ;
- accepter la licence Xcode ;
- lancer un simulateur iOS.

Validation : `flutter doctor -v` ne signale aucun blocage pour la cible choisie et `flutter devices` affiche au moins un appareil.

Le modèle mental de Flutter

PRINCIPE DÉCLARATIF

`UI = f(state)`

À chaque état correspond une interface.

État courant

Les données décrivent l'interface à afficher.

→

`build()`

Flutter traduit cet état en widgets.

→

Arbre de widgets

Les widgets composent la structure de l'écran.

→

Interface affichée

Le rendu visible reflète l'état courant.

Quand l'état change, Flutter relance `build()` et reconstruit uniquement la partie concernée de l'interface.

Tout est composé de widgets

```
Scaffold(
  appBar: AppBar(title: const Text('Bonjour')),
  body: Center(
    child: Column(
      mainAxisSize: .min,
      children: [
        const Icon(Icons.waving_hand, size: 64),
        const Text('Bienvenue !'),
        const Text('Votre première interface Flutte
        ElevatedButton(
          onPressed: () {},
          child: const Text('Continuer'),
        ),
      ],
    ),
  ),
)
```

Chaque élément de cet arbre est un widget.



Stateless ou Stateful ?

StatelessWidget

Décrit l'interface à partir des données reçues.

```
paramètres → build() → interface
```

Il ne possède aucun état local persistant.

StatefulWidget

Est associé à un objet `State` qui persiste.

```
état local → setState() → build()
```

Utile pour un contrôleur, une animation ou une sélection locale.

> Les deux peuvent être reconstruits. La question est : le widget doit-il conserver un état local lié à son cycle de vie ?

Deux formes d’état

État local

Appartient à un widget et à son cycle de vie.

- onglet sélectionné ;
- champ de formulaire en cours ;
- animation ;
- élément développé ou replié.

État applicatif

Décrit les données et la situation de l'application.

- liste des capteurs ;
- utilisateur connecté ;
- connexion au broker ;
- mesures reçues et erreurs.

> L'état local reste proche du widget. Pour l'état applicatif, il faut décider où placer la logique.

Quand l’écran prend trop de responsabilités

La première tentation consiste à tout gérer directement dans l’écran :

```
Écran
├── mise en page
├── appel réseau
├── gestion des erreurs
├── cache
├── navigation
├── calcul des alertes
└── état de chargement
```

Ce code peut fonctionner, mais il devient difficile à faire évoluer et à tester.

C’est le problème auquel MVVM va répondre.

MVVM

Un pattern d'architecture de présentation

Model · View · ViewModel

Séparer les responsabilités pour mieux faire évoluer l'application.

Les responsabilités en un coup d'œil

View

Affiche l'état et transmet les actions de l'utilisateur.

ViewModel

Porte l'état, expose les actions et orchestre les opérations.

Model

Représente les données du domaine, sans dépendre des widgets.

Repository

Abstrait l'origine des données et les opérations disponibles.

> La View dialogue avec ses ViewModels. Les ViewModels dépendent des abstractions de Repository, jamais des widgets ni de leurs implémentations techniques.

Le flux MVVM

Model

Représenter les données du domaine

Définir le modèle de données

Le Model représente les données du domaine.

Pour SensorHub, un modèle doit être :

- indépendant des widgets et de la navigation ;
- immutable après sa création ;
- comparable par son contenu ;
- facile à copier avec une valeur différente ;
- sérialisable depuis et vers JSON.

Nous utiliserons **Freezed** pour générer ces comportements.

Le package Freezed

Freezed est un package Dart de génération de code.

À partir d'une déclaration annotée avec `@freezed`, il génère les comportements techniques attendus d'un modèle immutable.

Sans génération, une classe de données complète demande beaucoup de code :

- constructeur et propriétés finales ;
- `copyWith` ;
- opérateur `==` et `hashCode` ;
- `toString` ;
- `fromJson` et `toJson`, avec un générateur JSON.

Freezed génère l'immutabilité, `copyWith`, l'égalité et `toString`. Associé à `json_serializable`, il permet également de générer la sérialisation JSON.

Le modèle `Sensor` avec Freezed

```dart
import 'package:freezed_annotation/freezed_annotation.dart';

part 'sensor.freezed.dart';
part 'sensor.g.dart';

@freezed
abstract class Sensor with _$Sensor {
  const factory Sensor({
    required String id,
    required String name,
    required String room,
    required double temperature,
    required double humidity,
    @Default(false) bool isOnline,
  }) = _Sensor;

  factory Sensor.fromJson(Map<String, Object?> json) ⇒
      _$SensorFromJson(json);
}
```

Ce que Freezed nous apporte

```dart
final updatedSensor = sensor.copyWith(
  temperature: 24.8,
  isOnline: true,
);

final sensorFromMqtt = Sensor.fromJson(json);
final payload = sensorFromMqtt.toJson();
```

- `sensor` n'est jamais modifié ; une nouvelle instance est créée.
- Deux capteurs ayant les mêmes valeurs sont considérés comme égaux.
- Freezed s'appuie sur `json_serializable` pour générer le passage entre JSON et Dart.
- Les objets sont plus sûrs à partager entre Repository, ViewModel et View.

Le compromis de la génération

Avantages

La génération automatise le code technique.

- moins de code répétitif ;
- modèles immutables ;
- comportement uniforme ;
- sérialisation plus fiable.

Contraintes

Le projet doit intégrer une étape de génération.

- dépendances supplémentaires ;
- fichiers générés ;
- génération à relancer après chaque évolution du modèle ;
- messages d'erreur parfois moins directs au début.

Notre choix : utiliser Freezed, car les messages des capteurs vont évoluer pendant tout le module.

Les écrans attendus — MVP des 7 premières heures

DASHBOARD

```
SensorHub          ↻

Mes capteurs

  Salon        21,8°
  45 %      En ligne

  Garage       29,6°
  61 %        Alerte
```

DÉTAIL D'UN CAPTEUR

```
‹ Capteur du salon

● En ligne

Température   21,8°C
Humidité         45 %
Batterie          82 %

État               OK

Dernière mesure
```

Le wireframe fixe les informations attendues. Les couleurs et les composants restent libres.

L'historique, l'authentification et l'ajout d'un capteur seront intégrés dans les séances suivantes.

Étape 1 — L’interface statique

- créer le projet Flutter ;
- créer le modèle immutable `Sensor` avec Freezed ;
- ajouter `json_serializable` et lancer la génération ;
- stocker provisoirement trois capteurs directement dans le widget ;
- afficher ces capteurs dans une liste ;
- extraire un widget `SensorCard` ;
- distinguer visuellement un capteur hors ligne.

Terminé lorsque

L’application démarre et affiche correctement la liste des capteurs.